

Here is the complete, all-in-one execution handbook for your BDLT lab practical exam. Every single problem statement from your sheet has been fully solved with its respective code, along with **exact step-by-step instructions on how to compile, deploy, and execute** them using Remix IDE and Truffle.

Part 1: Installation & Workspace Setup

[Task 3] Truffle Installation Steps

1. **Install Node.js:** Download and run the LTS installer from nodejs.org.
2. **Install Truffle Globally:** Open your terminal (Command Prompt/PowerShell on Windows, or Terminal on Mac/Linux) and run:

Bash

```
npm install -g truffle
```

3. **Verify Installation:** Run the following command to ensure it's successfully installed:

Bash

```
truffle version
```

[Task 5] MetaMask Installation Steps

1. **Download Extension:** Navigate to metamask.io or open your browser's extension store (Chrome Web Store).
2. **Add to Browser:** Click "**Add to Chrome**" (or your respective browser extension button) to install.
3. **Wallet Creation:** Click "**Create a new wallet**", agree to the terms, and set a secure local password.
4. **Backup Secret Phrase:** Reveal, write down, and safely store the 12-word **Secret Recovery Phrase**. Complete the verification step by selecting the words in the correct sequence.

Part 2: Truffle Local Blockchain Deployment

[Task 4] Simple Storage Program

1. The Code (SimpleStorage.sol)

Solidity

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract SimpleStorage {
    uint256 public storedData;

    function set(uint256 x) public {
        storedData = x;
    }

    function get() public view returns (uint256) {
        return storedData;
    }
}
```

2. Deployment Script (2_deploy_contracts.js)

JavaScript

```
const SimpleStorage = artifacts.require("SimpleStorage");

module.exports = function (deployer) {
    deployer.deploy(SimpleStorage);
};
```

3. Step-by-Step Truffle Deployment Instructions

1. **Initialize Project:** Create an empty folder, open your terminal inside it, and initialize a Truffle project:

Bash

```
truffle init
```

2. **Save Files:** * Save the SimpleStorage.sol file inside the generated contracts/ directory.

- Save the 2_deploy_contracts.js file inside the generated migrations/ directory.

3. **Launch Local Blockchain:** Run the built-in development blockchain tool:

Bash

truffle development

(This will open a local network console running on http://127.0.0.1:9545 and provide 10 test accounts).

4. **Compile and Migrate:** Inside the active interactive Truffle terminal prompt (truffle(development)>), execute:

Bash

compile

migrate

5. **Verify Execution:** Interact with your deployed contract directly via the console using:

JavaScript

```
let instance = await SimpleStorage.deployed();
await instance.set(42);
let value = await instance.get();
value.toString(); // Output: '42'
```

Part 3: Advanced Solidity Contracts (Remix IDE Execution)

For the following contracts, use **Remix IDE** (<https://remix.ethereum.org>) for the fastest and cleanest execution in a practical exam setting.

General Remix Deployment Steps (Applies to Tasks 10 through 47)

1. **Create File:** Open Remix IDE. In the File Explorer tab, click the **File icon** under contracts/ and give it a name (e.g., Voting.sol). Paste the code.
2. **Compile:** Click the **Solidity Compiler** tab (the third icon down on the sidebar). Ensure the compiler version matches your code (0.8.x). Click "**Compile [FileName].sol**".
3. **Choose Environment:** Click the **Deploy & Run Transactions** tab (the fourth icon down). Under **Environment**, choose "**Remix VM (Cancun)**" (this provides an instant, simulated blockchain browser environment with 10 pre-funded test accounts).
4. **Deploy:** Click the orange "**Deploy**" button.
5. **Interact:** Look under the "**Deployed Contracts**" section at the bottom left. Click the dropdown arrow next to your contract to expose input fields and clickable execution buttons.

[Task 10] Decentralized Voting System

- **Exam Requirement:** One vote per address; admin controls the start and end of the voting timeline.

Solidity

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract DecentralizedVoting {
    struct Candidate {
        string name;
        uint256 voteCount;
    }

    address public admin;
    bool public votingActive;
    Candidate[] public candidates;
    mapping(address => bool) public hasVoted;

    modifier onlyAdmin() {
        require(msg.sender == admin, "Only admin can perform this action");
        _;
    }

    constructor() {
        admin = msg.sender;
        votingActive = false;
    }

    function addCandidate(string memory _name) public onlyAdmin {
        candidates.push(Candidate({name: _name, voteCount: 0}));
    }

    function startVoting() public onlyAdmin {
        votingActive = true;
    }

    function endVoting() public onlyAdmin {
```

```

        votingActive = false;
    }

    function vote(uint256 _candidateIndex) public {
        require(votingActive, "Voting period is not active");
        require(!hasVoted[msg.sender], "You have already voted");
        require(_candidateIndex < candidates.length, "Invalid candidate
index");

        candidates[_candidateIndex].voteCount++;
        hasVoted[msg.sender] = true;
    }

    function getVoteCount(uint256 _candidateIndex) public view returns
(uint256) {
        return candidates[_candidateIndex].voteCount;
    }
}

```

Execution Steps in Remix:

1. Compile and click **Deploy**. Your current active account becomes the admin.
2. Expand the deployed contract panel. Enter a name in the input box next to addCandidate and click the button. Do this a couple of times to add candidate choices (index 0, index 1, etc.).
3. Click the **startVoting** button.
4. Select a different account from the top account dropdown menu to simulate a normal voter.
5. Enter 0 in the input field next to the blue **vote** button and click it.
6. Try clicking **vote** again with the same account—it will fail with an error revert ("You have already voted").
7. Switch back to the first account (the admin) and click **endVoting**. Any subsequent attempts to vote from any account will now fail.

[Task 14] Crowdfunding Platform

- **Exam Requirement:** If funding goal is missed before deadline, donors can execute a refund pull request.

Solidity

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract Crowdfunding {
    address public admin;
    uint256 public goal;
    uint256 public deadline;
    uint256 public totalRaised;
    mapping(address => uint256) public contributions;

    constructor(uint256 _goal, uint256 _durationInSeconds) {
        admin = msg.sender;
        goal = _goal;
        deadline = block.timestamp + _durationInSeconds;
    }

    function donate() public payable {
        require(block.timestamp < deadline, "Campaign has ended");
        require(msg.value > 0, "Contribution must be greater than 0");

        contributions[msg.sender] += msg.value;
        totalRaised += msg.value;
    }

    function withdrawFunds() public {
        require(block.timestamp >= deadline, "Campaign is still ongoing");
        require(totalRaised >= goal, "Funding goal was not met");
        require(msg.sender == admin, "Only admin can withdraw total raised funds");

        payable(admin).transfer(address(this).balance);
    }

    function refund() public {
        require(block.timestamp >= deadline, "Campaign is still ongoing");
        require(totalRaised < goal, "Goal was met, refunds unavailable");
    }
}
```

```

uint256 amountToRefund = contributions[msg.sender];
require(amountToRefund > 0, "No contributions found to refund");

contributions[msg.sender] = 0;
payable(msg.sender).transfer(amountToRefund);
}
}

```

Execution Steps in Remix:

1. In the text box next to the Deploy button, enter the initial constructor parameters.
For example: 10000000000000000000 (which is 10 Ether in Wei) and 60 (60 seconds deadline). Click **Deploy**.
2. Select a non-admin account. To pay funds, set the **Value** input field near the top of the Remix panel to 2 and the unit dropdown to **Ether**.
3. Click the red **donate** button.
4. **Scenario A (Success):** If total donations cross the 10 Ether limit before 60 seconds pass, switch back to the admin account after 60 seconds have elapsed and click **withdrawFunds**.
5. **Scenario B (Refund):** If the 60 seconds pass and you did not meet the 10 Ether limit, select the donor account and click **refund** to pull back your contributed Ether.

[Task 18] Decentralized To-Do List

- **Exam Requirement:** Tasks are mapped independently per calling wallet and kept private from third parties.

Solidity

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract TodoList {
    struct Task {
        string content;
        bool completed;
    }

    mapping(address => Task[]) private userTasks;
}

```

```

    function addTask(string memory _content) public {
        userTasks[msg.sender].push(Task({content: _content, completed:
false}));
    }

    function toggleCompleted(uint256 _index) public {
        require(_index < userTasks[msg.sender].length, "Task does not
exist");
        userTasks[msg.sender][_index].completed =
!userTasks[msg.sender][_index].completed;
    }

    function getMyTasks() public view returns (Task[] memory) {
        return userTasks[msg.sender];
    }
}

```

Execution Steps in Remix:

1. Compile and click **Deploy**.
2. Under the deployed contract interactions, enter "Buy Groceries" inside the field next to **addTask** and click the button.
3. Click **getMyTasks** to view your list arrays showing index position, string content, and a false completed boolean status.
4. Enter 0 in the field next to **toggleCompleted** and click it.
5. Click **getMyTasks** again; the completed property will now show as true.
6. Switch to a completely different user account from the Remix dropdown list. Click **getMyTasks**—the list will return completely empty, proving privacy isolation.

[Task 22] Simple Bank (Wallet System)

- **Exam Requirement:** Simple operations (Deposit, Withdraw, Check balance) restricted to account holder.

Solidity

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract SimpleBank {

```

```

mapping(address => uint256) private balances;

function deposit() public payable {
    require(msg.value > 0, "Cannot deposit 0 ether");
    balances[msg.sender] += msg.value;
}

function withdraw(uint256 _amount) public {
    require(balances[msg.sender] >= _amount, "Insufficient balance");

    balances[msg.sender] -= _amount;
    payable(msg.sender).transfer(_amount);
}

function checkBalance() public view returns (uint256) {
    return balances[msg.sender];
}
}

```

Execution Steps in Remix:

1. Compile and click **Deploy**.
2. Select any user account. Go to the top **Value** box, type 5, select **Ether** as the denomination unit, and hit the dark red **deposit** button.
3. Click the blue **checkBalance** button. It will output 5000000000000000000 (5 Ether converted to Wei units).
4. To withdraw, enter 2000000000000000000 (2 Ether in Wei) into the input box next to the **withdraw** function and click it.
5. Click **checkBalance** again to verify your bank balance has correctly dropped to 3 Ether.

[Task 27] Supply Chain Management

- **Exam Requirement:** Append status logs tracking changes with automated block timestamps and sender logs.

Solidity

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

```

```

contract SupplyChain {
    struct TrackLog {
        string status;
        uint256 timestamp;
        address handler;
    }

    struct Product {
        uint256 id;
        string name;
        TrackLog[] history;
    }

    mapping(uint256 => Product) private products;
    uint256 public productCount;

    function createProduct(string memory _name) public {
        productCount++;
        Product storage newProduct = products[productCount];
        newProduct.id = productCount;
        newProduct.name = _name;

        newProduct.history.push(TrackLog({
            status: "Manufactured",
            timestamp: block.timestamp,
            handler: msg.sender
        }));
    }

    function updateStatus(uint256 _productId, string memory _newStatus)
public {
        require(_productId <= productCount && _productId > 0, "Product
mismatch");

        products[_productId].history.push(TrackLog({
            status: _newStatus,
            timestamp: block.timestamp,
            handler: msg.sender

```

```

    }));
}

function getProductHistory(uint256 _productId) public view returns
(TrackLog[] memory) {
    require(_productId <= productCount && _productId > 0, "Product
mismatch");
    return products[_productId].history;
}
}

```

Execution Steps in Remix:

1. Compile and click **Deploy**.
2. Type "Laptop" inside **createProduct** and execute the transaction. This registers the item at product ID 1.
3. Switch accounts to simulate a distributor agent. Enter 1 as the ID and "In Transit" inside **updateStatus** and execute the transaction.
4. Switch accounts again to simulate the final delivery address. Enter 1 and "Delivered" inside **updateStatus** and execute.
5. Click **getProductHistory** with an input value of 1. It will output the complete historical timeline array containing the status changes, matching addresses, and block timestamps.

[Task 31] NFT Minting Contract

- **Exam Requirement:** Creation, tracked transfers, and custom metadata uniform bindings.

Solidity

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract SimpleNFT {
    string public name = "Digital Asset NFT";
    string public symbol = "DANFT";

    uint256 public nextTokenId;
    mapping(uint256 => address) public tokenOwner;
}

```

```

mapping(uint256 => string) public tokenURIs;

function mint(string memory _metadataURI) public {
    uint256 currentId = nextTokenId;
    tokenOwner[currentId] = msg.sender;
    tokenURIs[currentId] = _metadataURI;
    nextTokenId++;
}

function transfer(address _to, uint256 _tokenId) public {
    require(tokenOwner[_tokenId] == msg.sender, "You do not own this token");
    require(_to != address(0), "Invalid recipient address");

    tokenOwner[_tokenId] = _to;
}
}

```

Execution Steps in Remix:

1. Compile and click **Deploy**.
2. Under **mint**, enter a sample metadata URI text string like "ipfs://my_crypto_art_link" and execute. This creates Token ID 0 owned by your current address.
3. Check ownership status by entering 0 into the input field for the blue **tokenOwner** button—it will show your address.
4. Copy a different wallet address from the top Remix accounts dropdown. Switch back to the first account, enter that copied address alongside token ID 0 into the **transfer** fields (_to, _tokenId), and click transaction execution.
5. Call **tokenOwner** for ID 0 again to verify ownership has updated to the new destination wallet address.

[Task 35] Subscription Service

- **Exam Requirement:** Timed verification validation access; automatic expiration.

Solidity

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

```

```

contract SubscriptionService {
    uint256 public constant SUBSCRIPTION_COST = 0.01 ether;
    uint256 public constant DURATION = 30 days;

    mapping(address => uint256) public subscriptionExpires;

    function subscribe() public payable {
        require(msg.value == SUBSCRIPTION_COST, "Incorrect subscription
fee");

        if (block.timestamp > subscriptionExpires[msg.sender]) {
            subscriptionExpires[msg.sender] = block.timestamp + DURATION;
        } else {
            subscriptionExpires[msg.sender] += DURATION;
        }
    }

    function hasAccess(address _user) public view returns (bool) {
        return block.timestamp < subscriptionExpires[_user];
    }
}

```

Execution Steps in Remix:

1. Compile and click **Deploy**.
2. Select any account and call **hasAccess** using that account address. It will output false.
3. Set your Remix environment **Value** block to 0.01 and set the unit dropdown to **Ether**. Click **subscribe**.
4. Now, check **hasAccess** again for that same user—the output changes to true.
5. Check the raw expiration timestamp value inside the **subscriptionExpires** mapping to confirm it is exactly 30 days ahead of the block time.

[Task 39] Escrow Payment System

- **Exam Requirement:** Double agreement clearance parameters or automated payout rules post-timeout.

Solidity

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract EscrowSystem {
    address public buyer;
    address public seller;
    uint256 public releaseDeadline;
    bool public buyerApproved;
    bool public sellerApproved;
    bool public fundsReleased;

    constructor(address _seller, uint256 _disputePeriodSeconds) payable {
        buyer = msg.sender;
        seller = _seller;
        releaseDeadline = block.timestamp + _disputePeriodSeconds;
        fundsReleased = false;
    }

    function approveRelease() public {
        if (msg.sender == buyer) buyerApproved = true;
        if (msg.sender == seller) sellerApproved = true;

        if (buyerApproved && sellerApproved) {
            executePayout();
        }
    }

    function claimTimeoutPayout() public {
        require(block.timestamp >= releaseDeadline, "Deadline hasn't
passed");
        executePayout();
    }

    function executePayout() private {
        require(!fundsReleased, "Funds already claimed");
        fundsReleased = true;
        payable(seller).transfer(address(this).balance);
    }
}
```

```
}  
}
```

Execution Steps in Remix:

1. Select **Account 2** from the list and copy its address (this will be the seller).
2. Switch to **Account 1** (the buyer). In the Value field near the top, configure an escrow deposit amount of 5 **Ether**.
3. In the text boxes next to the Deploy button, paste the copied seller address and set the dispute timer to 120 seconds. Click **Deploy**. (The contract now holds the 5 Ether securely).
4. Click **approveRelease** using Account 1.
5. Switch to Account 2 (the seller) and click **approveRelease**. The 5 Ether will be instantly paid out to the seller's wallet balance.
6. (*Alternative Path*): If both parties don't explicitly approve, wait for 120 seconds to pass, then call **claimTimeoutPayout** to release the locked funds automatically.

[Task 43] Decentralized Lottery System

- **Exam Requirement:** Multi-user participant entry tickets with pseudo-random choice resolution.

Solidity

```
// SPDX-License-Identifier: MIT  
pragma solidity ^0.8.0;  
  
contract DecentralizedLottery {  
    address public manager;  
    address[] public players;  
  
    constructor() {  
        manager = msg.sender;  
    }  
  
    function buyTicket() public payable {  
        require(msg.value == 0.01 ether, "Ticket price is exactly 0.01  
Ether");  
        players.push(msg.sender);  
    }  
}
```

```

    }

    function pickWinner() public {
        require(msg.sender == manager, "Only the manager can pick the winner");
        require(players.length > 0, "No players in the pool");

        uint256 randomIndex =
uint256(keccak256(abi.encodePacked(block.timestamp, block.prevrandao,
players))) % players.length;
        address winner = players[randomIndex];

        players = new address[](0); // Reset pool
        payable(winner).transfer(address(this).balance); // Transfer pool
funds
    }
}

```

Execution Steps in Remix:

1. Compile and click **Deploy** using **Account 1** (the manager).
2. Switch to **Account 2**, change the top **Value** selector field to 0.01 **Ether**, and click **buyTicket**.
3. Repeat this step using **Account 3** and **Account 4** so there are multiple unique addresses stored in the participant tracking array.
4. Switch back to **Account 1** (the manager) and click the **pickWinner** button.
5. Check the account balances of your players. One of them will have won the total combined ticket pool distribution.

[Task 47] Certificate Issuance and Verification

- **Exam Requirement:** Cryptographic identity mapping records; validation query lookup via IDs.

Solidity

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract CertificateVerifier {

```

```

struct Certificate {
    string candidateName;
    string courseName;
    address recipient;
    uint256 dateOfIssuance;
    bool isValid;
}

address public authority;
mapping(bytes32 => Certificate) private certificates;

constructor() {
    authority = msg.sender;
}

function issueCertificate(
    string memory _id,
    string memory _name,
    string memory _course,
    address _recipient
) public {
    require(msg.sender == authority, "Only authority can issue certificates");
    bytes32 idHash = keccak256(abi.encodePacked(_id));
    require(!certificates[idHash].isValid, "Certificate ID already exists");

    certificates[idHash] = Certificate({
        candidateName: _name,
        courseName: _course,
        recipient: _recipient,
        dateOfIssuance: block.timestamp,
        isValid: true
    });
}

function verifyCertificate(string memory _id) public view returns (
    string memory name, string memory course, address recipient, uint256
    date, bool valid

```

```

    } {
        bytes32 idHash = keccak256(abi.encodePacked(_id));
        Certificate memory cert = certificates[idHash];
        return (cert.candidateName, cert.courseName, cert.recipient,
cert.dateOfIssuance, cert.isValid);
    }
}

```

Execution Steps in Remix:

1. Compile and click **Deploy**. Your current address acts as the trusted authority.
2. Expand the **issueCertificate** input row. Enter matching values: “CERT123”, “Rahul”, “Blockchain BTech”, and any valid wallet address. Execute the transaction.
3. To verify, move down to the blue **verifyCertificate** text block field.
4. Input the identical string “CERT123” and press the button. It will output the name, course details, issuance date timestamp, and a true validation confirmation status flag.

Part 4: Web3 Frontend Program Integration

This completes the **Web3.js Program** requirement. This setup bridges an HTML user interface with a smart contract via MetaMask.

1. Save File (index.html)

Save the following code block as a file named index.html on your computer:

HTML

```

<!DOCTYPE html>
<html lang="en">
<head>
    <title>Web3 Lab Practice Execution</title>
    <script
src="https://cdn.jsdelivr.net/npm/web3@1.8.1/dist/web3.min.js"></script>
</head>
<body style="font-family: Arial, sans-serif; padding: 20px;">

    <h2>Web3 Lab Execution Gateway</h2>
    <button id="connectBtn" onclick="initWeb3()">Connect MetaMask</button>
    <p id="walletAddress">Status: Wallet Not Connected</p>

```

```

</hr>

<h3>Interact with Contract</h3>
<input type="number" id="inputData" placeholder="Enter Value to Set">
<button onclick="writeToBlockchain()">Send Data</button>
<button onclick="readFromBlockchain()">Fetch Current Data</button>
<p id="displayArea"></p>

<script>
    let web3Instance;
    let activeAccount;
    let deployedContract;

    // Note: For actual testing, replace these with your compiled
contract's details
    const contractAddress =
"0x5346067000000000000000000000000000000000";
    const contractABI = [
        { "inputs": [{ "internalType": "uint256", "name": "x", "type":
"uint256" }], "name": "set", "outputs": [], "stateMutability": "nonpayable",
"type": "function" },
        { "inputs": [], "name": "get", "outputs": [{ "internalType":
"uint256", "name": "", "type": "uint256" }], "stateMutability": "view",
"type": "function" }
    ];

    async function initWeb3() {
        if (window.ethereum) {
            try {
                web3Instance = new Web3(window.ethereum);
                // Requests wallet connection from MetaMask
                const accounts = await window.ethereum.request({ method:
'eth_requestAccounts' });
                activeAccount = accounts[0];
                document.getElementById("walletAddress").innerText =
"Connected to: " + activeAccount;

                deployedContract = new
web3Instance.eth.Contract(contractABI, contractAddress);

```

```

        } catch (error) {
            console.error("User denied account access");
        }
    } else {
        alert("MetaMask extension not discovered. Please install
it.");
    }
}

async function writeToBlockchain() {
    const numericValue = document.getElementById("inputData").value;
    if(!deployedContract) return alert("Connect your wallet setup
first!");

    document.getElementById("displayArea").innerText = "Processing
transaction hash...";
    // Triggers a MetaMask transaction pop-up signature request
    await deployedContract.methods.set(numericValue).send({ from:
activeAccount });
    document.getElementById("displayArea").innerText = "Transaction
confirmed successfully!";
}

async function readFromBlockchain() {
    if(!deployedContract) return alert("Connect your wallet setup
first!");

    const outcome = await deployedContract.methods.get().call();
    document.getElementById("displayArea").innerText = "Value pulled
from contract state: " + outcome;
}
</script>
</body>
</html>

```

2. Step-by-Step Web3 UI Execution Instructions

1. **Deploy Contract:** Deploy your SimpleStorage.sol contract using Remix, but under **Environment**, select **"Injected Provider - MetaMask"**. (Confirm the connection prompt inside your MetaMask popup extension). Click **Deploy**.

2. **Copy Artifacts:** Once deployed, copy the **Contract Address** from the deployed contracts tab, and copy the **ABI** from the Solidity compiler metadata section. Paste them into the designated placeholder variables inside your local index.html file.
3. **Run a Local Server:** You cannot just double-click an HTML file locally because browsers block direct wallet API injections from file:// paths. Open the folder in VS Code, right-click your file, and choose **"Open with Live Server"** (or run a quick local host terminal server via `python -m http.server 8000`).
4. **Interact:** Open `http://127.0.0.1:5500` or `http://localhost:8000` in your web browser.
5. Click **"Connect MetaMask"**. Your wallet extension will open asking you to authorize connection permissions.
6. Enter a value (e.g., 99) in the text box and click **"Send Data"**. MetaMask will pop up asking you to review and sign the gas fee transaction request. Confirm it, wait for the transaction to complete, and then click **"Fetch Current Data"** to read it directly from the blockchain state.

The Universal Frontend (`universal_tester.html`)

Save this code block as a single file. It dynamically rebuilds the interface on the fly based on whichever contract you paste into it.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <title>Universal BDLT Web3 Controller</title>
  <script
src="https://cdn.jsdelivr.net/npm/web3@1.8.1/dist/web3.min.js"></script>
  <style>
    body { font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
padding: 30px; background-color: #f4f6f9; color: #333; }
    .card { background: white; padding: 20px; border-radius: 8px; box-
shadow: 0 4px 6px rgba(0,0,0,0.1); margin-bottom: 20px; }
    textarea, input { width: 100%; padding: 10px; margin: 8px 0; border:
1px solid #ccc; border-radius: 4px; box-sizing: border-box; }
```

```

        button { background-color: #007bff; color: white; border: none;
padding: 10px 20px; border-radius: 4px; cursor: pointer; font-weight: bold;
}

        button:hover { background-color: #0056b3; }
        .status { font-weight: bold; color: #28a745; margin: 10px 0; }
    </style>
</head>
<body>

    <h2>Universal Blockchain Lab Interface</h2>

    <div class="card">
        <h3>1. Gateway Authorization</h3>
        <button onclick="connectWallet()">Connect MetaMask Wallet</button>
        <p id="walletStatus" class="status">Status: Disconnected</p>
    </div>

    <div class="card">
        <h3>2. Contract Alignment</h3>
        <label>Target Contract Address:</label>
        <input type="text" id="targetAddress" placeholder="0x...">

        <label>Target Contract ABI (Copy from Remix Compiler
Metadata):</label>
        <textarea id="targetABI" rows="5"
placeholder='[{"inputs": [], "name": "myFunction", ...}]'></textarea>

        <button onclick="loadContract()">Attach & Load Contract</button>
        <p id="contractStatus" class="status">Status: No active contract
loaded</p>
    </div>

    <div class="card">
        <h3>3. Low-Level Execution Panel</h3>
        <p>To execute functions without a custom UI, enter the precise
function signature and parameter inputs below:</p>

        <label>Function Name (e.g., <code>vote</code> or
<code>deposit</code>):</label>

```

```

    <input type="text" id="funcName" placeholder="vote">

    <label>Arguments / Inputs (Comma-separated if multiple, leave blank
if none):</label>
    <input type="text" id="funcArgs" placeholder="0">

    <label>Ether Value (Optional - Use only for <code>payable</code>
functions like deposit/donate):</label>
    <input type="text" id="ethValue" placeholder="0.01">

    <button onclick="executeWrite()">Send Transaction (Write)</button>
    <button onclick="executeRead()" style="background-color:
#28a745;">Query State (Read)</button>

    <hr/>
    <h4>Execution Output Log:</h4>
    <p id="consoleLog" style="font-family: monospace; background: #eee;
padding: 10px; border-radius: 4px; min-height: 40px; white-space: pre-
wrap;">Awaiting user instruction...</p>
  </div>

  <script>
    let web3;
    let account;
    let activeContract;

    async function connectWallet() {
      if (window.ethereum) {
        web3 = new Web3(window.ethereum);
        const accounts = await window.ethereum.request({ method:
'eth_requestAccounts' });
        account = accounts[0];
        document.getElementById("walletStatus").innerText =
"Connected Wallet Address: " + account;
      } else {
        alert("MetaMask extension not found!");
      }
    }
  }

```

```

function loadContract() {
    const addr =
document.getElementById("targetAddress").value.trim();
    const abiString =
document.getElementById("targetABI").value.trim();

    if (!addr || !abiString) return alert("Please supply both an
address and an ABI string.");

    try {
        const parsedABI = JSON.parse(abiString);
        activeContract = new web3.eth.Contract(parsedABI, addr);
        document.getElementById("contractStatus").innerText =
"Successfully mapped target contract at: " + addr;
        document.getElementById("consoleLog").innerText = "Ready to
handle transactions.";
    } catch (err) {
        alert("ABI parsing failure. Ensure you copied valid JSON
formatting.");
    }
}

async function executeWrite() {
    if (!activeContract) return alert("Load a target contract
first!");
    const method = document.getElementById("funcName").value.trim();
    const argsString =
document.getElementById("funcArgs").value.trim();
    const ethVal = document.getElementById("ethValue").value.trim();

    const args = argsString ? argsString.split(',') : [];
    document.getElementById("consoleLog").innerText = "Awaiting
signature confirmation from MetaMask...";

    try {
        let txOptions = { from: account };
        if (ethVal && parseFloat(ethVal) > 0) {
            txOptions.value = web3.utils.toWei(ethVal, 'ether');
        }
    }
}

```

```

        const recipe = await
activeContract.methods[method](...args).send(txOptions);
        document.getElementById("consoleLog").innerText =
"Transaction Settled!\nHash: " + recipe.transactionHash;
    } catch (err) {
        document.getElementById("consoleLog").innerText = "Execution
Error:\n" + err.message;
    }
}

async function executeRead() {
    if (!activeContract) return alert("Load a target contract
first!");
    const method = document.getElementById("funcName").value.trim();
    const argsString =
document.getElementById("funcArgs").value.trim();
    const args = argsString ? argsString.split(',') : [];

    document.getElementById("consoleLog").innerText = "Querying
blockchain data storage states...";

    try {
        const result = await
activeContract.methods[method](...args).call();
        document.getElementById("consoleLog").innerText = "State
Read Result:\n" + JSON.stringify(result, null, 2);
    } catch (err) {
        document.getElementById("consoleLog").innerText = "Query
Error:\n" + err.message;
    }
}
</script>
</body>
</html>

```